AWS Application Load Balancer (ALB) and Auto Scaling Group (ASG)

Step-by-Step Setup and Troubleshooting Runbook

Document type: Technical Runbook
Environment: AWS EC2 Learning Environment
Audience: Beginners and Cloud Support Engineers
Purpose: Configure a Load Balancer and Auto Scaling so an application can run on multiple EC2 instances and automatically add or remove instances based on demand.

---

1. Objective

This runbook explains how to configure an AWS Application Load Balancer and Auto Scaling Group for an application running on EC2.

The objective is to:

- Distribute application traffic across multiple servers.
- Automatically add application servers when demand increases.
- Automatically reduce the number of servers when demand decreases.
- Check application server health.
- Understand how to troubleshoot unhealthy targets.
- Learn the setup through the AWS Management Console.

Important: This runbook is for a learning environment. Do not apply it directly to a production environment without appropriate testing, change approval, backup and rollback planning.

2. Scenario and Assumptions

The initial environment contains one EC2 instance running the application and database.

For this exercise, we assume the database is hosted separately so that multiple application servers can connect to the same database.

Item	Configuration
Cloud provider	Amazon Web Services (AWS)
Compute service	Amazon EC2
Application protocol	HTTP
Application port	8080
Public access	Domain name
Load balancer	Application Load Balancer
Load balancer listener	HTTP, port 80
Target group	`learning-app-tg`
Launch template	`learning-app-template`
Auto Scaling Group	`learning-app-asg`
Desired capacity	1
Minimum capacity	1
Maximum capacity	2
Database	Separate database server
Scaling method	CPU-based scaling, configured in a later step

The existing domain initially points to the EC2 instance's public IP address. We will change the domain to point to the Load Balancer only after the new setup is tested.

3. Architecture Overview

3.1 Existing architecture

Users → Domain → EC2 public IP → Application on port 8080

The application and database originally run on one server.

3.2 Target architecture

Users → Domain → Application Load Balancer → Application EC2 instances → Shared database

The Load Balancer distributes requests across healthy application servers.

The Auto Scaling Group manages the number of application servers.

All application servers connect to the same database so that users work with consistent data.

4. Prerequisites

Before starting, confirm the following:

- Access to the AWS Management Console.
- An existing VPC and suitable subnets.
- An application that runs successfully on port 8080.
- A separate database that the application can access.
- An application server image (AMI) that can start the application correctly.
- A suitable security group for the Load Balancer.
- A suitable security group for the application servers.
- A database security group that permits connections from the application servers.
- An ACM certificate if HTTPS will be configured on the Load Balancer.

An AMI must contain a working application setup. Simply copying an image of the original server may also copy its database, local files or other unwanted configuration. Prepare and test an application-only image before using it for Auto Scaling.

5. Procedure 1 — Create the Target Group

5.1 Purpose

A target group is a collection of application servers that the Load Balancer can send requests to.

It also checks whether the application servers are healthy.

5.2 Configuration steps

1. Sign in to the AWS Management Console.

2. Open the EC2 service.
3. In the left navigation menu, select **Target Groups** under Load Balancing.
4. Click **Create target group**.
5. Enter the following settings.

Setting	Value
Target type	Instances
Target group name	`learning-app-tg`
Protocol	HTTP
Port	8080
VPC	Same VPC as the Load Balancer
Health check protocol	HTTP
Health check path	`/`, if it returns a successful response

6. Click **Next**.
7. Do not register the original server if it has not been prepared as an application-only server.
8. Click **Create target group**.

5.3 Verification

Open the target group and check that `learning-app-tg` exists.

The target group may initially have no registered targets. This is expected if no application-only server has been registered yet.

6. Procedure 2 — Create the Application Load Balancer

6.1 Purpose

The Application Load Balancer is the entry point for web traffic.

Instead of sending requests directly to an EC2 public IP, users connect to the Load Balancer, which forwards requests to healthy application servers.

6.2 Create the Load Balancer

1. Open **EC2** → **Load Balancers**.
2. Click **Create load balancer**.
3. Select **Application Load Balancer**.
4. Enter the following details.

Setting	Value
Load balancer name	`learning-app-alb`
Scheme	Internet-facing
IP address type	IPv4
VPC	Same VPC as the application servers

5. Under network mappings, select at least two suitable public subnets in different Availability Zones.
6. Confirm that the public subnets have a route to an Internet Gateway.

6.3 Configure the Load Balancer security group

Create or select a security group that allows inbound web traffic.

Type	Port	Source
HTTP	80	`0.0.0.0/0`
HTTPS	443	`0.0.0.0/0`

Port 443 is needed when HTTPS is configured. The HTTPS listener still requires an appropriate TLS certificate.

6.4 Configure the listener

1. Under **Listeners and routing**, configure HTTP on port 80.
2. Set the default action to forward traffic to `learning-app-tg`.
3. Review the settings.
4. Click **Create load balancer**.

6.5 Verification

Open **EC2** → **Load Balancers**.

Select `learning-app-alb` and confirm that its state becomes **Active**.

At this point, the Load Balancer can exist without successfully serving the application. A healthy registered application target is required.

7. Procedure 3 — Prepare the Launch Template

7.1 Purpose

A Launch Template is a reusable configuration that tells AWS how to launch an EC2 instance.

The Auto Scaling Group uses this template whenever it needs another application server.

7.2 Create the Launch Template

1. Open **EC2** → **Launch Templates**.
2. Click **Create launch template**.
3. Enter the following details.

Setting	Value
Launch template name	`learning-app-template`
Description	Application server for Auto Scaling lab
AMI	Prepared application-only AMI
Instance type	Suitable for the application

Key pair	Select a key pair if required for administration
Security group	Application server security group

4. Review the configuration.
5. Click **Create launch template**.

7.3 Application server security group

The application server must allow incoming traffic from the Load Balancer on port 8080.

Type	Port	Source
Custom TCP	8080	Load Balancer security group

Do not open port 8080 to the entire internet merely to make the health check work.

7.4 Database connectivity

The database security group must allow connections on the database's actual port from the application server security group.

Common database ports include:

- MySQL/MariaDB: 3306
- PostgreSQL: 5432
- Microsoft SQL Server: 1433

Use the correct port for the actual database. Do not open the database port to the public internet.

8. Procedure 4 — Create the Auto Scaling Group

8.1 Purpose

An Auto Scaling Group manages a group of EC2 instances.

For example, if the desired capacity is one instance, AWS tries to maintain one running application server. When a scaling policy requires more capacity, AWS can launch another instance, up to the configured maximum.

8.2 Create the group

1. Open **EC2** → **Auto Scaling Groups**.
2. Click **Create Auto Scaling group**.
3. Enter:
   `learning-app-asg`
4. Select the launch template `learning-app-template`.
5. Select the appropriate template version.
6. Click **Next**.

8.3 Choose the network

1. Select the same VPC as the Load Balancer.
2. Select suitable application subnets in at least two Availability Zones.
3. Prefer private subnets for application servers when the required network access is available.

8.4 Attach the Load Balancer

1. Select the option to attach to an existing Load Balancer.
2. Choose the option to select an existing target group.
3. Select `learning-app-tg`.
4. Enable Elastic Load Balancing health checks where available.
5. Continue to the next step.

8.5 Set the capacity

For this learning exercise, use:

Setting	Value
Desired capacity	1
Minimum capacity	1
Maximum capacity	2

This means AWS should maintain one application server initially and can scale out to two when the scaling policy requires it.

8.6 Complete the creation

1. Continue through the remaining configuration screens.
2. Review the settings.
3. Create the Auto Scaling Group.

8.7 Verification

Open `learning-app-asg`.

Confirm that the group has the expected desired capacity and that the EC2 instance launches successfully.

Then open:

EC2 → Target Groups → `learning-app-tg` → Targets

Wait for the target status to become **Healthy**.

9. Procedure 5 — Configure Automatic Scaling

Creating an Auto Scaling Group does not automatically mean it will scale based on CPU. A scaling policy must be configured.

9.1 Create a target tracking policy

1. Open **EC2 → Auto Scaling Groups**.
2. Select `learning-app-asg`.
3. Open the **Automatic scaling** tab.
4. Choose **Create dynamic scaling policy** or the equivalent option in the current console.
5. Select **Target tracking scaling policy**.
6. Enter a policy name, such as:
   `learning-cpu-target-tracking`
7. Select the predefined metric **Average CPU utilization**.
8. Set the target value to `50` percent for the learning exercise.
9. Save the policy.

The target value is an example for learning, not a universal production recommendation.

9.2 How it works

- When average CPU utilization is above the target, AWS may add an instance.
- When demand decreases, AWS may remove an instance, subject to the policy and scaling safeguards.
- The group will not exceed its maximum capacity of two instances.
- The group will not scale below its minimum capacity of one instance.

Scaling takes time. A new instance must launch, start the application and pass its health checks before it can serve traffic.

Important: CPU-based scaling helps only when CPU is a meaningful indicator of application load. If slowness comes from the database, memory, disk I/O or application locks, CPU scaling alone may not fix it.

10. Procedure 6 — Verify the End-to-End Setup

Perform the following checks before directing users to the new Load Balancer.

10.1 Check the Load Balancer

Open **EC2** → **Load Balancers** → **`learning-app-alb`**.

Confirm:

- State is Active.
- Listener configuration is correct.
- The default action forwards to `learning-app-tg`.
- The configured subnets and security group are correct.

10.2 Check the target group

Open **Target Groups** → **`learning-app-tg`** → **Targets**.

Confirm that the application instances are Healthy.

10.3 Check the application

Open the Load Balancer's DNS name in a browser using HTTP for the initial test.

For example:

`http://<load-balancer-dns-name>`

Replace the placeholder with the DNS name shown in the AWS Console.

Confirm that the expected application page loads and that the application can connect to the shared database.

10.4 Check Auto Scaling

Open `learning-app-asg` and review:

- Desired capacity
- Minimum capacity
- Maximum capacity
- Current instances
- Activity history
- Scaling policy

Do not consider the exercise complete until a second instance has been tested successfully and both application instances can serve requests.

11. Procedure 7 — Configure HTTPS and the Domain

The Load Balancer must be tested before changing the existing domain.

11.1 Configure HTTPS

1. Open AWS Certificate Manager (ACM) in the same Region as the Load Balancer.
2. Request or locate a certificate for the domain.
3. Complete the required domain validation.
4. Open **EC2** → **Load Balancers** → **`learning-app-alb`**.
5. Add an HTTPS listener on port 443.
6. Select the validated ACM certificate.

7. Configure the listener to forward requests to `learning-app-tg`.

If HTTP should redirect to HTTPS, configure the port 80 listener with a redirect action instead of forwarding HTTP requests to the application.

11.2 Update the domain

After the new application environment is validated:

1. Open the DNS provider where the domain is managed.
2. Back up or record the current DNS configuration.
3. Point the domain to the Load Balancer using the appropriate DNS record.
4. For Route 53, an Alias record can point to the Load Balancer.
5. Verify that the domain resolves correctly and HTTPS works.

DNS propagation can take time. Keep the original server available until the new environment has been validated and the rollback plan is no longer needed.

12. Troubleshooting — Target Shows Unhealthy

An Unhealthy target means the Load Balancer's health check is failing.

Step 1 — Check the reason

Open:

EC2 → Target Groups → `learning-app-tg` → Targets

Inspect the health status details or reason.

Step 2 — Identify the error

Error	Likely cause	What to check
Request timed out	Network or firewall issue	Security groups, routes, network ACLs and whether the application responds
Connection refused	Nothing listening on the port or connection rejected	Application service and port 8080

HTTP 404	Incorrect health-check path	Use a valid health-check endpoint
HTTP 500	Application error	Application logs and database connectivity
HTTP 301 or 302	Redirect response	Health-check endpoint and success-code configuration

Step 3 — Check the security groups

The application server's inbound rules must allow TCP 8080 from the Load Balancer security group.

The Load Balancer must be able to send traffic to the application servers.

Step 4 — Check the application

On a Linux application server, an administrator can run:

```
sudo ss -lntp | grep ':8080'
```

This checks whether a process is listening on port 8080.

The application should be listening on a network interface reachable from the Load Balancer, not only on localhost.

Step 5 — Check the health-check path

Use an endpoint that returns an expected successful response without requiring user authentication.

For example, `/health` may be appropriate if the application provides that endpoint.

Do not change the success codes merely to hide an application error.

Step 6 — Wait and verify

After correcting the cause, allow time for subsequent health checks and confirm that the target becomes Healthy.

If the Auto Scaling Group repeatedly replaces an instance, inspect its activity history and fix the underlying problem.

13. Troubleshooting — Auto Scaling Does Not Add an Instance

If CPU utilization increases but the group does not scale:

1. Open **EC2** → **Auto Scaling Groups**.
2. Select `learning-app-asg`.
3. Check the Automatic scaling tab.
4. Confirm that the target tracking policy exists and is enabled.
5. Confirm that the maximum capacity is greater than the current capacity.
6. Check the CloudWatch CPU metric.
7. Check the Activity history for launch failures.
8. Confirm that AWS has sufficient EC2 capacity and that the launch template is valid.

For this exercise, maximum capacity is two. If two instances already exist, the group cannot scale beyond two.

14. Testing the Scale-Out and Scale-In Behaviour

Perform this test only in the learning environment.

Test A — Normal load

1. Confirm that one instance is running.
2. Confirm that the target is Healthy.
3. Open the application through the Load Balancer DNS name.

Expected result: The application works through the Load Balancer.

Test B — Increased load

1. Generate controlled test load on the application using an appropriate test method.
2. Monitor the CloudWatch CPU metric.
3. Open the Auto Scaling Group Activity history.
4. Observe whether a second instance launches.
5. Wait for the new instance to pass health checks.

Expected result: The group may scale out to two instances when the policy's conditions are met.

Do not generate heavy load against a real production system.

Test C — Reduced load

1. Stop the test workload.
2. Wait for CPU utilization to decrease.
3. Monitor the Auto Scaling Group.
4. Confirm that the group scales in if the policy determines that another instance is no longer needed.

Expected result: The group can return to one instance, subject to the scaling policy and its timing.

15. Rollback Plan

If the new setup does not work:

1. Do not terminate the original server.
2. Keep the original DNS record documented.
3. If DNS has been changed, restore the original DNS record when necessary.
4. Check the application and database connectivity.
5. Review target health, Load Balancer listeners and Auto Scaling activity.
6. Do not delete the original application or database until the new setup is fully validated.

Remember that DNS caching can delay a rollback.

16. Final Verification Checklist

- Target group `learning-app-tg` exists.
- Load Balancer `learning-app-alb` is Active.
- Load Balancer listener is configured correctly.
- Launch template `learning-app-template` uses a working application-only AMI.
- Auto Scaling Group `learning-app-asg` exists.
- Desired capacity is one.
- Minimum capacity is one.
- Maximum capacity is two.
- Scaling policy is configured.
- Application targets become Healthy.
- Application is accessible through the Load Balancer DNS name.
- All application servers connect to the same database.

- A second instance has been tested.
- HTTPS has been configured if required.
- The domain has been updated only after testing.
- The rollback procedure is documented.

17. Important Notes

1. A Load Balancer distributes traffic; it does not itself create additional servers.
2. An Auto Scaling Group creates and removes instances according to its capacity settings and scaling policies.
3. Health checks determine whether targets are ready to receive traffic.
4. The database must be shared by all application instances.
5. Local files and in-memory sessions must be handled appropriately when running multiple application servers.
6. An AMI should be tested before it is used as the source for Auto Scaling.
7. HTTPS requires an appropriate certificate and listener configuration.
8. Additional EC2 instances, the Load Balancer, data transfer and other resources may incur AWS charges.

Expected final result: Users access the application through the Load Balancer. The Auto Scaling Group manages the application instances, and each application instance uses the same shared database. The application can scale within the configured limits when demand changes.